

Table of Contents

Problem Statement	1
Phase 1: Problem Understanding & Industry Analysis.....	3
Phase 2: Org Setup & Configuration Summary	6
Phase 3: Data Modeling & Implementation Report.....	10
Phase 4: Process Automation Implementation Final Report	14
Phase 5: Apex Programming Implementation Report	23

Campus Recruitment Command Center

An Integrated System for Student Placements & Corporate Relations

PAWAN KUMAR RAJAK

Problem Statement

Project Title: "Campus Recruitment Command Center - An Integrated System for Student Placements & Corporate Relations"

Industry: Higher Education

Project Type: B2S (Business-to-Student) & B2B (Business-to-Business) Salesforce Platform

Target Users: Training & Placement (T&P) Officers, Students, and the Head of Placements

Problem Statement

A university's Training & Placement (T&P) cell manages hundreds of corporate relationships and thousands of student applications using disconnected tools like spreadsheets, Google Forms, and email. This manual process results in communication gaps, missed application deadlines for students, and an inability for T&P officers to efficiently match the right students with the right opportunities. Furthermore, the constant influx of student queries via email overwhelms the T&P staff, leading to slow response times for critical questions.

To address this, the university wants to implement a Salesforce platform to:

- Centralize company and job posting information.

- Automate the student application and notification process.
- Streamline and automate the resolution of common student queries.
- Provide real-time dashboards for the Head of Placements to track success metrics.

Use Cases

1. Corporate & Recruiter Management

- Maintain a unified database of all visiting companies and their recruiter contact information.
- Track the status of engagement with new companies (e.g., Approached, MoU Signed, Visit Scheduled).
- Log all interactions, such as calls and meetings, with company representatives.

2. Job Posting & Application Management

- Create and manage detailed Job Postings with specific eligibility criteria (e.g., Branch, GPA, Graduation Year).
- Automatically notify eligible students via email when a new relevant job is posted.
- Allow students to apply for jobs through a simple, guided interface (a Screen Flow).
- Track the status of each student's application (e.g., Applied, Shortlisted, Offer Extended).

3. Automated Student Query Resolution

- Automatically create a "Query" record in Salesforce when a student sends an email to a designated placement helpdesk address (e.g., placements@university.edu).
- Send an automated initial email response acknowledging receipt of the query.
- Use a Flow to scan the query's subject for keywords (e.g., "Resume," "Eligibility," "Deadline") and send back a templated email with helpful links or standard answers, closing common queries without manual intervention.

4. Reporting & Analytics

- A master dashboard for the Head of Placements showing key metrics like:
 - Number of Companies Visiting
 - Total Job Openings vs. Applications
 - Branch-wise Placement Percentage
- Reports that track the placement funnel and T&P officer activity.

Phase 1: Problem Understanding & Industry Analysis

Date: September 10, 2025

1. Requirement Gathering

The primary goal of this project is to transform the university's manual and fragmented placement process into a streamlined, automated, and centralized system using the Salesforce platform.

Functional Requirements (What the system must do):

- **Company & Recruiter Management:** A central database to store and manage information about visiting companies (Accounts) and their recruiters (Contacts).
- **Job Posting Management:** T&P officers must be able to create, update, and manage job postings with specific eligibility criteria (e.g., Branch, GPA, required skills).
- **Automated Student Notifications:** The system must automatically email eligible students as soon as a new, relevant job is posted.
- **Student Application Portal:** Students need a simple, guided way to view and apply for open positions.
- **Application Tracking:** The system must track the status of each student's application (Applied, Shortlisted, Interviewing, Offer Extended, Rejected).
- **Automated Query Handling:** The system must automatically create a case/record from incoming student emails and provide an automated, templated response for common queries.
- **Reporting & Analytics:** The Head of Placements requires real-time dashboards to view key metrics like placement rates, company engagement, and application numbers.

Technical Requirements:

- The solution must be built on the Salesforce Lightning Platform.
- The system must enforce data security through a role-based model (T&P Officer vs. Student).
- The solution should be scalable to handle data for thousands of students and hundreds of companies.

2. Stakeholder Analysis

The success of the project depends on meeting the needs of its key users.

Stakeholder Role	Needs & Goals	Current Pain Points
T&P Officer	- Quickly add new companies and jobs. Reduce manual data entry and email tasks. Track all student applications in one place.	- Data is scattered across multiple spreadsheets. Manually filtering and emailing students is time-consuming and error-prone. Overwhelmed by repetitive student questions.
Student	- A single, reliable source for all job openings. Timely alerts for jobs they are eligible for. A clear and simple application process.	- Fear of missing deadlines due to communication gaps. Confusion from information on different platforms (email, WhatsApp, website). Slow response times for important queries.
Head of Placements	- High-level, real-time view of placement statistics. Data-driven insights to improve corporate relations. Professional reports for university management.	- Lack of accurate, real-time data for strategic decision-making. Reporting is a manual and time-consuming process at the end of the placement season.

3. Business Process Mapping

As-Is Process (Current State):

1. A company recruiter emails job details to a T&P Officer.
2. The T&P Officer manually checks a master student spreadsheet to find eligible candidates.
3. The Officer composes and sends a bulk email to the filtered list.
4. Students apply by replying to the email with their resumes.
5. The T&P Officer tracks these applications in another spreadsheet.
6. This process is slow, inefficient, and creates multiple versions of the truth.

To-Be Process (Future State with Salesforce):

1. A T&P Officer creates a Job Posting record in Salesforce with defined eligibility criteria.
2. **SYSTEM ACTION:** A Record-Triggered Flow instantly sends a templated email to all Contact records (Students) who meet the criteria.
3. The student clicks a link in the email, launching a Screen Flow to apply.
4. **SYSTEM ACTION:** Upon flow completion, an Application record is automatically created, linking the Student and the Job Posting.
5. All data is instantly available on a real-time dashboard for the Head of Placements.

4. Industry-specific Use Case Analysis (Higher Education)

In the Higher Education sector, the Training & Placement cell is a critical function that directly impacts the university's reputation and ranking. This project addresses a core industry challenge: bridging the gap between academia and the corporate world effectively.

- **Enhancing the Student Experience:** A smooth, transparent, and professional placement process is a key differentiator for universities. This system directly enhances the student experience at the most critical final stage of their journey.
- **Strengthening Corporate Partnerships:** Providing corporate partners with a streamlined recruiting process makes the university a more attractive place to hire from, leading to better and more numerous job opportunities for students.
- **Data-Driven Strategy:** This system moves the T&P cell from a purely administrative function to a strategic one, allowing the Head of Placements to use data to identify trends and improve their outreach strategies.

5. AppExchange Exploration

A review of the Salesforce AppExchange was conducted to identify existing solutions.

- **Salesforce Education Cloud:** This is Salesforce's flagship product for the education industry. It is an extensive, enterprise-level solution that manages the entire student lifecycle, from admissions to alumni relations. While powerful, it is too large and complex for the specific, immediate needs of the T&P cell and the 5-day project scope.
- **Third-Party Apps:** Various apps exist for event management, email marketing, and form building.

Conclusion: A solution that is perfectly tailored to the university's unique placement process, can be developed rapidly, and avoids the cost and complexity of a large, pre-built package. This project will deliver maximum value by focusing directly on solving the T&P cell's most pressing problems.

Phase 2: Org Setup & Configuration Summary

Date: September 11, 2025

1. Core Company Configuration

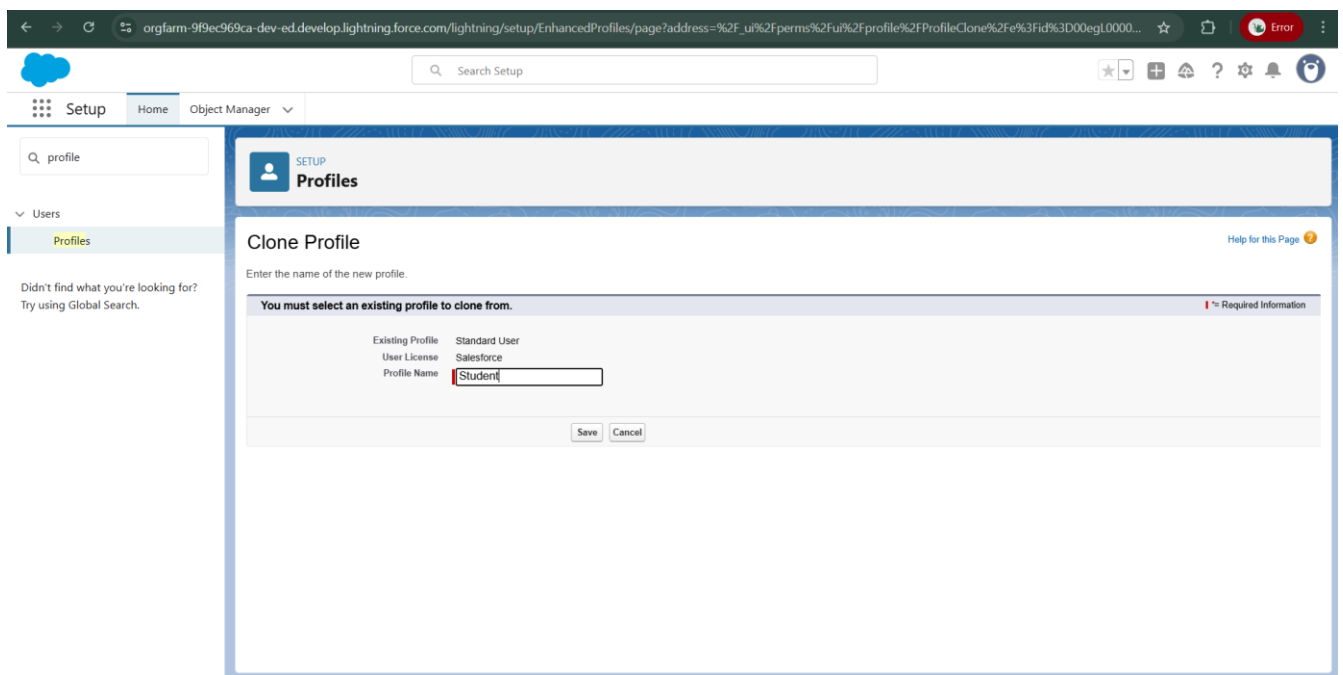
The foundational settings for the Salesforce org were established to mirror the operational environment of the university's T&P Cell.

- **Company Information:** The org was configured with the university's name, default time zone (Asia/Kolkata), and locale to ensure all time-stamped data is accurate.
- **Business Hours & Fiscal Year:** Standard business hours (Mon-SAT, 12 AM - 12 AM) were set to enable future tracking of case/query response times. A standard fiscal year was confirmed for reporting consistency.
- **Developer Org:** A new Salesforce Developer Edition org was created to serve as the dedicated environment for this project.

2. Security Model Implementation

A multi-layered security model was configured to ensure data is accessible only to the appropriate users.

- **Profiles:** Two custom profiles were created by cloning the "Standard User" profile:
 - T&P Officer: For staff who will manage the system.
 - Student: For end-users who will interact with the system.



The screenshot displays the Salesforce Setup interface. The left sidebar shows the 'Setup' menu with 'Profiles' selected under the 'Users' section. The main content area is titled 'Clone Profile' and prompts the user to 'Enter the name of the new profile.' Below this, a message states 'You must select an existing profile to clone from.' A table lists the 'Existing Profile' as 'Standard User' and the 'User License' as 'Salesforce'. The 'Profile Name' field is populated with 'Student'. At the bottom of the dialog, there are 'Save' and 'Cancel' buttons. The browser's address bar shows the URL: 'orgfarm-9f9ec969ca-dev-ed.develop.lightning.force.com/lightning/setup/EnhancedProfiles/page?address=%2F_u%2Fperms%2Fu%2Fprofile%2FProfileClone%2Fe%3Fid%3D00egL0000...'

Setup Home Object Manager

Q profiles

Users

Profiles

Didn't find what you're looking for? Try using Global Search.

SETUP Profiles

Profiles

All Profiles Edit Delete Create New View

New Profile

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Other All

Action	Profile Name	User License
Edit Del ...	Student	Salesforce
Edit Clone	System Administrator	Salesforce
Edit Del ...	T&P Officer	Salesforce
Edit Clone	Work.com Only User	Work.com Only

1-45 of 45 0 Selected Previous Next Page 1 of 1

- **Role Hierarchy:** A top-down data visibility structure was created: **CEO -> Head of Placements -> T&P Officer**. This ensures management can view the records of their subordinates.

Setup Home Object Manager

Q roles

Users

Roles

Feature Settings

Sales

Contact Roles on Contracts

Contact Roles on Opportunities

Service

Case Teams

Case Team Roles

Contact Roles on Cases

Didn't find what you're looking for? Try using Global Search.

SETUP Roles

Creating the Role Hierarchy

You can build on the existing role hierarchy shown on this page. To insert a new role, click **Add Role**.

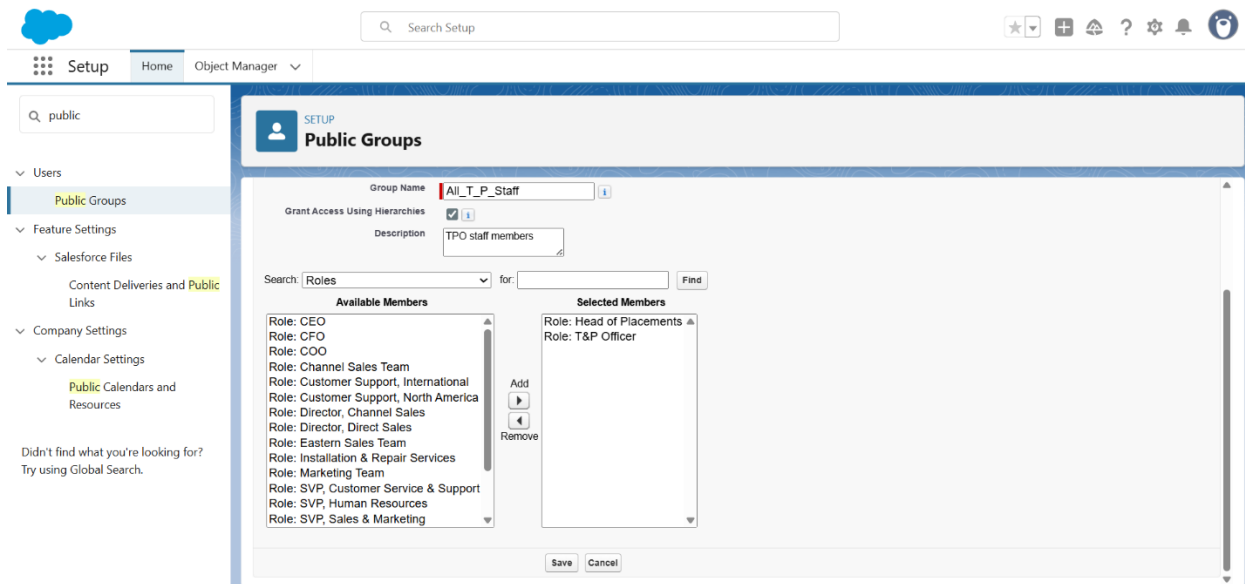
Your Organization's Role Hierarchy

Collapse All Expand All

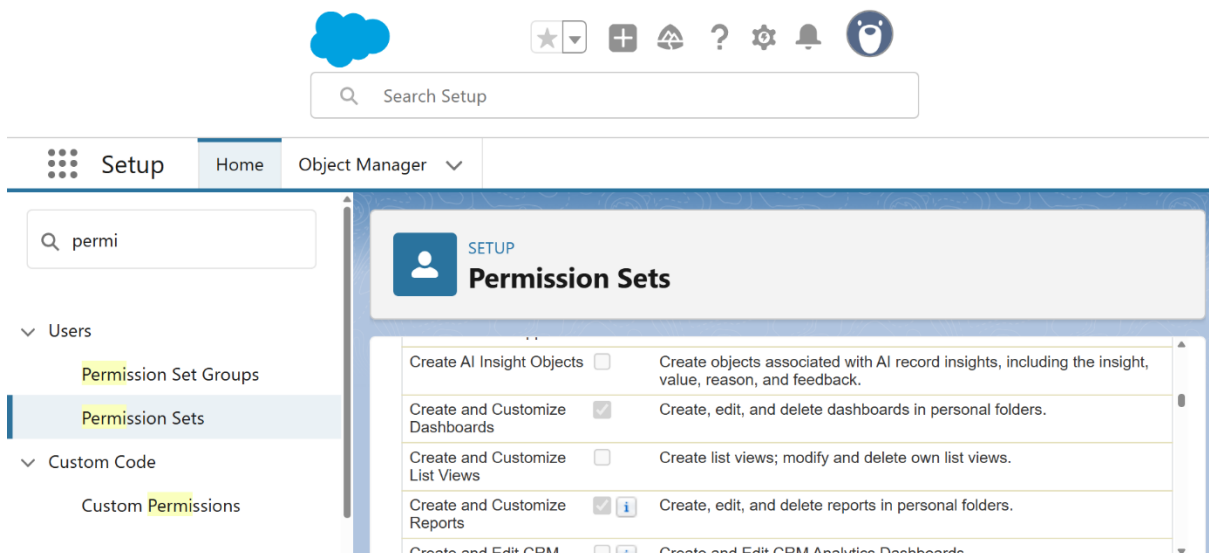
- Gyan Ganga Institute of Technology and Sciences
 - Add Role
 - CEO Edit | Del | Assign
 - Add Role
 - CFO Edit | Del | Assign
 - Add Role
 - COO Edit | Del | Assign
 - Add Role
 - Head of Placements Edit | Del | Assign
 - Add Role
 - T&P Officer Edit | Del | Assign
 - Add Role
 - SVP Customer Service & Support Edit | Del | Assign
 - Add Role
 - SVP Human Resources Edit | Del | Assign
 - Add Role
 - SVP Sales & Marketing Edit | Del | Assign
 - Add Role

Show in tree view

- **Organization-Wide Defaults (OWD):** A **Private** data model was established as the baseline. This is a security best practice, ensuring users can only see their own records by default unless explicitly granted access.



- Permission Set:** A Permission Set named T&P Analytics Manager was created to grant specific, additive permissions (e.g., "Create and Customize Dashboards") to select users without changing their entire profile. This demonstrates a flexible approach to security.



3. Deferred Configurations & Justification

Certain configurations mentioned in the project checklist were intentionally deferred to a later phase for strategic reasons.

- **Sharing Rules:** Sharing Rules are used to open up access to records that are locked down by a Private OWD. These rules are record-specific. **Reason for Deferral:** The custom objects (Job Posting, Application, T&P Query) have not been created yet. Sharing Rules for these objects will be configured in Phase 4 after the data model is built.
- **Login IP Ranges:** This security feature restricts user logins to specific IP addresses. **Reason for Deferral:** During the development phase, it is impractical to restrict IP ranges as development may occur from various locations. This is a production-level security measure that would be implemented post-development, before final deployment.
- **Sandbox Usage & Deployment:** These concepts relate to the development lifecycle in a team environment. **Reason for Deferral:** As a rapid, 5-day solo project, development is being done directly in a Developer Org for maximum efficiency. In a real-world team project, a sandbox-driven development and deployment strategy would be utilized.

Phase 3: Data Modeling & Implementation Report

Date: September 12, 2025

1. Overview

This document outlines the actions taken during the Data Modeling phase of the project. The objective was to create a robust, scalable, and secure data structure to support all functionalities of the Training & Placement (T&P) application. This involved configuring standard Salesforce objects, creating new custom objects and fields, establishing relationships, and enhancing the user interface.

2. Standard Object Configuration

Standard Salesforce objects were leveraged and configured to fit the project's specific terminology and data segregation needs.

- **Account Object:**
 - **Action:** The Account object was renamed to Company.
 - **Justification:** This provides a more intuitive user experience for T&P Officers, who primarily manage relationships with companies, not "accounts."
- **Contact Object (with Record Types):**
 - **Action:** Two record types were created on the Contact object: Student and Recruiter. Each record type was assigned a unique page layout (Student Layout and Recruiter Layout).
 - **Justification:** The Contact object stores two distinct types of people. Record Types are the Salesforce best practice for segregating them, allowing for different data fields and user experiences on the same object.
- **Lead Object:**
 - **Action:** The standard Lead object was not renamed or configured for this project's initial scope.
 - **Justification:** The primary focus is on managing relationships with established corporate partners (Companies) and student applications. While future enhancements could use Leads for tracking potential new corporate partners, it is not part of the core requirement for this 5-day build.

3. Custom Object & Field Creation

Three core custom objects were created to form the backbone of the application.

Object 1: Job Posting (Job_Posting__c)

Object 1: Job Posting (Job_Posting__c)

Field Label	API Name	Data Type	Details
Job Posting Name	Name	Text	The primary identifier/title of the job.
Company	Company__c	Master-Detail	A required link to the parent Company (Account) record.
Description	Description__c	Text Area (Long)	For the detailed job description.
Location	Location__c	Text	The city/location of the job.
Package (CTC)	Package_CTC__c	Currency	The offered Cost-to-Company.
Eligibility Branch	Eligibility_Branch__c	Picklist (Multiselect)	Values: Computer Science, Mechanical, Electronics, Civil.
Minimum GPA	Minimum_GPA__c	Number (2,1)	Required GPA for eligibility.
Status	Status__c	Picklist	Values: Open, Interviewing, Closed.

Object 2: Application (Application__c) – Junction Object

Field Label	API Name	Data Type	Details
Application ID	Name	Auto-Number	Format: APP-{0000}.
Job Posting	Job_Posting__c	Master-Detail	A required link to the parent Job_Posting__c record.
Student	Student__c	Master-Detail	A required link to the parent Contact (Student) record.
Status	Status__c	Picklist	Values: Applied, Shortlisted, Interview Scheduled, Offer Extended, Rejected.

Object 3: T&P Query (T_P_Query__c)

Field Label	API Name	Data Type	Details
Query ID	Name	Auto-Number	Format: Q-{00000}.
Student	Student__c	Lookup	An optional link to the Contact (Student) record.
Description	Description__c	Text Area (Long)	The full text of the student's question.
Status	Status__c	Picklist	Values: New, Automated Reply Sent, Manual Follow-up Needed, Closed.
Category	Category__c	Picklist	Values: Eligibility, Resume, Deadline, Technical Issue, Other.

4. Relationship Decisions (Master-Detail vs. Lookup)

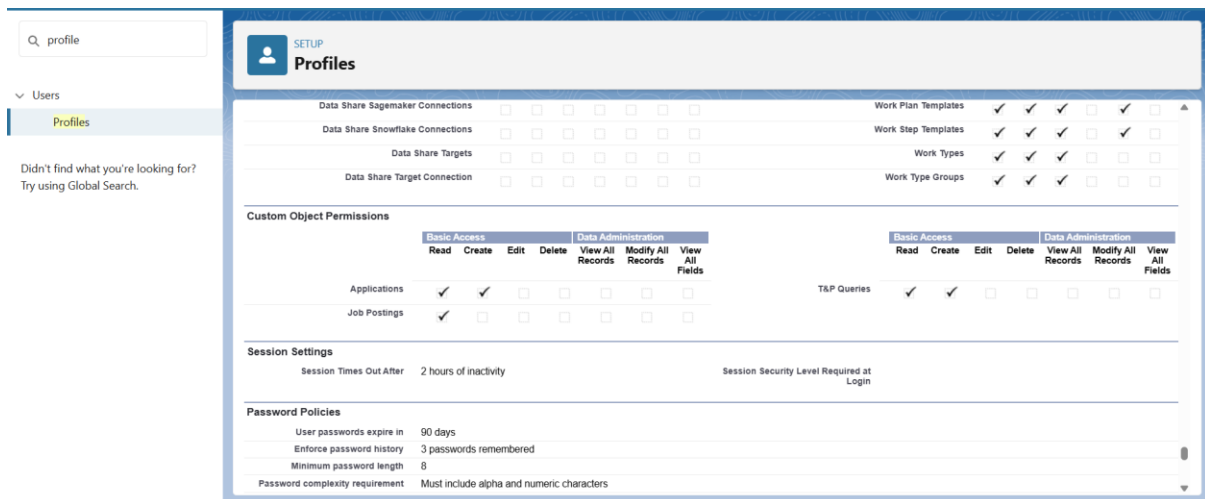
The choice of relationship type is critical for data integrity.

- **Master-Detail Relationship (MDR):** Used on the Application object for its links to Job Posting and Contact.
 - **Justification:** An application is fundamentally dependent on both a job and a student. It cannot exist on its own. The MDR enforces this by ensuring an application record is automatically deleted if either its associated job or student is deleted (cascading delete). This also allows for rollup summary fields in the future.
- **Lookup Relationship:** Used on the T&P Query object to link to the Contact.
 - **Justification:** A query is related to a student, but it is also a historical record for the T&P cell. If a student's record is deleted after graduation, the T&P cell may still want to retain the query for analytics. A Lookup provides a looser connection that prevents the query from being auto-deleted, thus preserving historical data.

5. User Interface & Field-Level Security

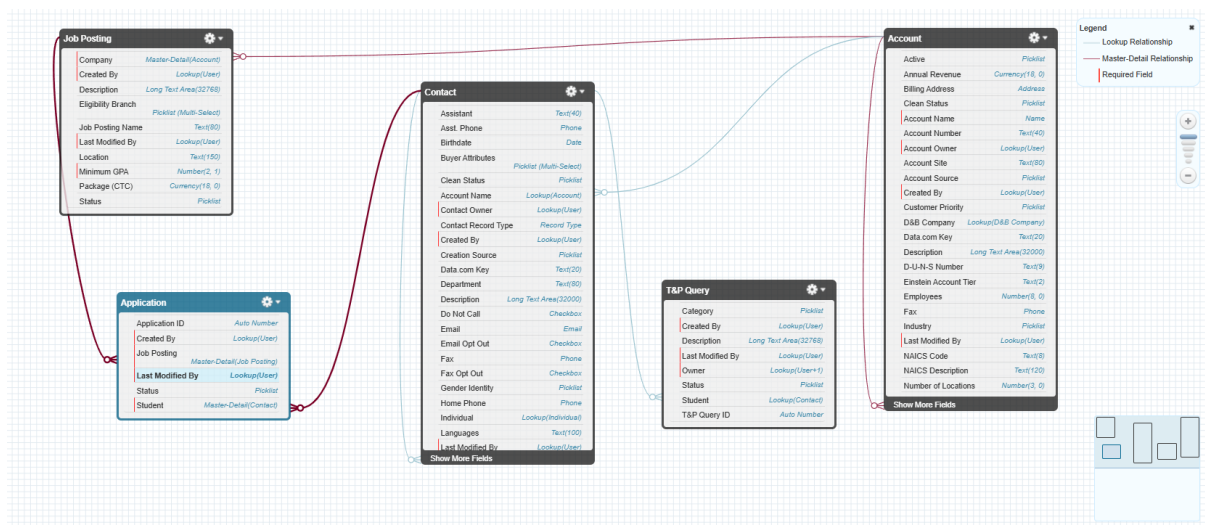
- **Page Layouts & Compact Layouts:** Custom page layouts were created for the Student and Recruiter record types. A custom Compact Layout, Job Posting Highlights, was created to show key information at the top of the job posting page.

- **Field-Level Security (FLS):** FLS was configured to ensure users only see relevant data.
 - **User-Driven Modification:** Based on user requirements for transparency, the Package (CTC)___c field on the Job Posting object has been made **Read-Only** for the Student profile. This allows students to make informed decisions while preventing them from editing the data.
 - **Internal Fields:** The Category___c field on the T&P Query object was **hidden** from the Student profile as it is an internal classification field for T&P officers only.



6. Schema Builder Visualization

The Schema Builder was used to view the complete data model. This provides a clear, visual blueprint of all objects and their intricate relationships, confirming the logical integrity of the application's architecture.



7. Excluded Concepts

- **Hierarchical & External Objects:** These relationship types were not applicable. Hierarchical relationships are specific to the User object, and External Objects are for integrating data from outside systems, which is beyond the scope of this self-contained project.

Phase 4: Process Automation Implementation Final Report

Date: September 17, 2025

1. Overview

This document provides a comprehensive summary of the process automation implemented during Phase 4. Building upon the secure data model from the previous phases, the focus was to embed the T&P Cell's business logic directly into the application. This was achieved by creating a suite of automations that enforce data quality, guide user actions, eliminate manual work, and ensure timely, professional communication. All automations were built using Salesforce Flow, adhering to modern best practices for scalability and maintenance.

2. Implemented Automations & Processes

A. Validation Rule: Preventing Data Inconsistency

A validation rule was implemented to protect data integrity and enforce a critical business process.

- **Business Need:** The T&P Cell cannot officially close a Job Posting if there are still students with active applications for that role. This prevents administrative errors and ensures no student is left in limbo.
- **Implementation Steps:**
 1. A **Roll-Up Summary Field** named Active Application Count was created on the Job Posting object. This field provides a real-time count of all related Application records that are not in a final state (Offer Extended or Rejected).

Setup > OBJECT MANAGER

Job Posting

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

Aggregates

☒ COUNT

☐ SUM

☐ MIN

☐ MAX

Field to Aggregate:

Filter Criteria

☐ All records should be included in the calculation

☒ Only records meeting certain criteria should be included in the calculation

Field	Operator	Value	
Status	not equal to	Offer Extended	AND
Status	not equal to	Rejected	AND
--None--	--None--		AND
--None--	--None--		AND
--None--	--None--		

For checkbox fields, enter a value of True for checked or False for not checked. For picklist fields, enter the master picklist field value in your corporate language.

Previous Next Cancel

2. A **Validation Rule** was then created on the Job Posting object. This rule checks two conditions upon saving a record: if the Status is being changed to "Closed" AND if the Active Application Count is greater than 0. If both are true, the save action is blocked, and a clear error message is displayed to the user.

Setup > OBJECT MANAGER

Job Posting

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Job Posting Validation Rule

Define a validation rule by specifying an error condition and a corresponding error message. The error condition is written as a Boolean formula expression that returns true or false. When the formula expression returns true, the save will be aborted and the error message will be displayed. The user can correct the error and try again.

Validation Rule Edit Save Save & New Cancel

Rule Name:

Active: ☒

Description:

Error Condition Formula

Example: `Discount_Percent__c > 0.30` More Examples...

Display an error if Discount is more than 30%

If this formula expression is **true**, display the text defined in the Error Message area

Insert Field Insert Operator

```

AND(
  ISPICKVAL(Status__c, "Closed"),
  Active_Application_Count__c > 0
)

```

Functions

-- All Function Categories --

ABS

ACOS

ADDMONTHS

AND

ASCII

ASIN

Insert Selected Function

ABS(number)

Returns the absolute value of a number, a number without its sign

Quick Tips

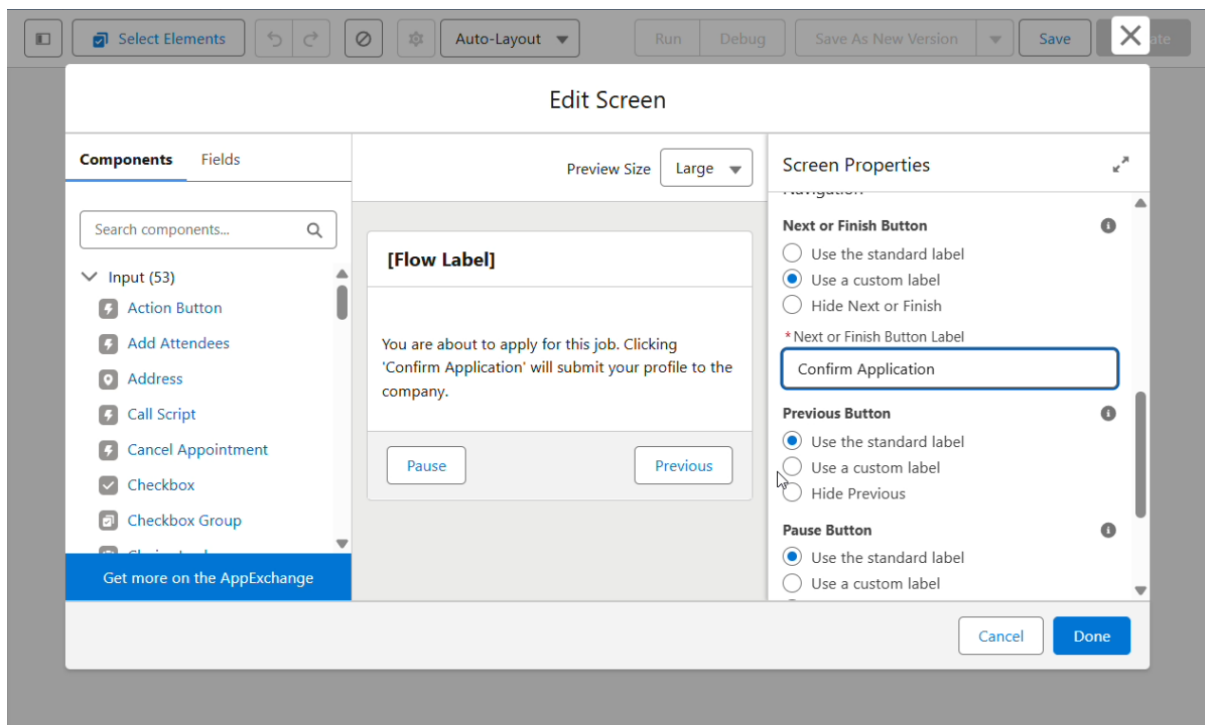
- Operators & Functions

! = Required Information

B. Screen Flow: A Guided Application Experience for Students

To enhance the student user experience and streamline the application process, a guided screen flow was developed.

- **Business Need:** Students require a simple, intuitive, and error-proof method to apply for jobs. A manual process of creating records would be inefficient and confusing.
- **Implementation Steps:**
 1. A **Screen Flow** titled "Student Application Process" was built. It features a simple confirmation screen to ensure the student's intent.
 2. The flow is intelligent, automatically capturing the recordId of the Job Posting it was launched from and the ContactId of the logged-in student user (\$User.ContactId).
 3. Upon confirmation, a Create Records element automatically generates a new Application record, correctly linking the student and job, and setting the initial Status to "Applied".
 4. This flow was then made accessible by creating a **Quick Action** button labeled "Apply Now" and adding it to the Job Posting page layout, ensuring high visibility.



The screenshot displays the Salesforce Flow Builder interface for a flow named "Student Application Process - V1". The flow diagram on the left shows a sequence of steps: "Screen Flow Start", "Application Confirmation Screen", "Create Application (Create Records)", and "End". The "Create Application" step is highlighted, and its configuration is shown on the right.

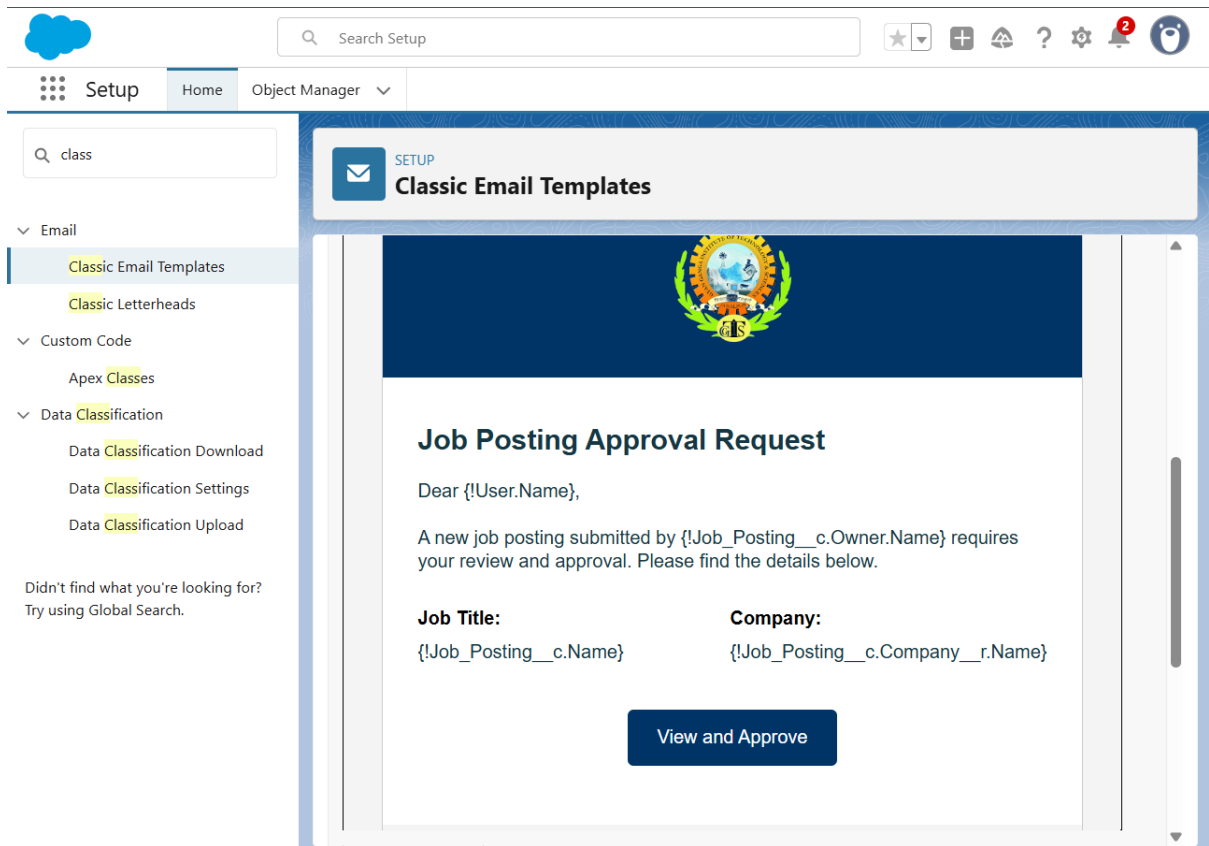
The configuration for the "Create Records" step is as follows:

- Trigger:** Manually
- Create a Record of This Object:** Application
- Set Field Values for the Application:**
 - Field: Job Posting → Value: recordId
 - Field: Student → Value: Running User > ContactId
 - Field: Status → Value: Applied
- Buttons:** + Add Field
- Checkbox:** ☐ Manually assign variables (advanced)
- Check for Matching Records:** (unchecked)

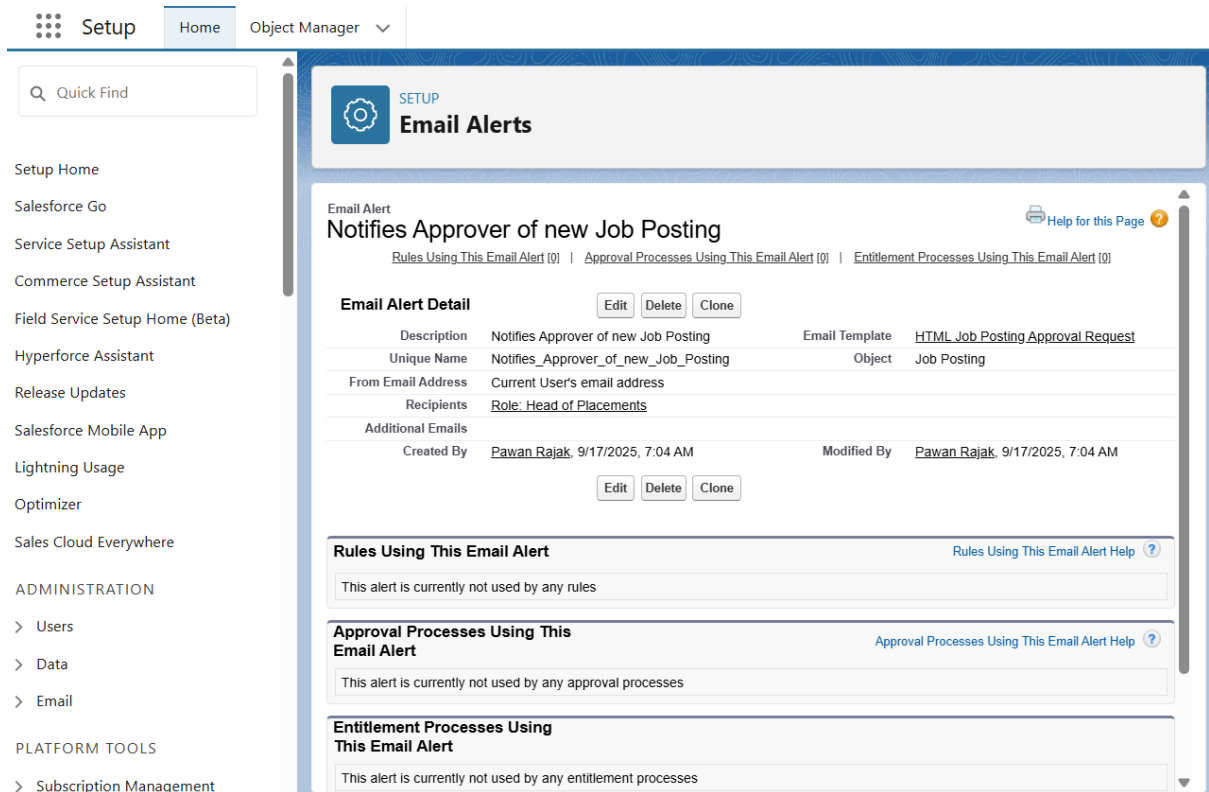
C. Approval Process & HTML Email Alert: Formalizing Job Postings

A formal approval process, supported by a professional HTML email alert, was created to ensure management oversight.

- **Business Need:** All new job postings must be reviewed and approved by the Head of Placements before they are made public to students, ensuring quality and consistency. The notification for this approval must be professional and actionable.
- **Implementation Steps:**
 1. An **HTML Email Template** was created to provide a branded, easy-to-read notification. The template includes the university logo, merges in key details like the Job Title and Company, and features a prominent "View and Approve" button that links directly to the record in Salesforce.



2. An **Email Alert** action was configured to use this HTML template and send it to any user with the Head of Placements role.



3. An **Approval Process** was configured on the Job Posting object. It triggers when a record's Status is set to "Submitted for Approval".

The screenshot shows the Salesforce Setup interface. The left sidebar has a search bar with 'appro' and a list of categories: Data, Feature Settings, and Process Automation. Under 'Process Automation', 'Approval Processes' is selected. The main content area is titled 'Approval Processes' and shows the configuration for 'Job Posting: Job Posting Approval'. The 'Process Definition Detail' section includes fields for Process Name, Unique Name, Description, Entry Criteria, Record Editability, and Approval Assignment. The 'Initial Submitters' field is set to 'Account Owner'. The 'Created By' field is 'Pawan Rajak' and the 'Modified By' field is 'Pawan Rajak'.

Process Definition Detail	
Process Name	Job Posting Approval
Unique Name	Job_Posting_Approval
Description	
Entry Criteria	Job Posting: Status EQUALS Submitted for Approval
Record Editability	Administrator ONLY
Approval Assignment	Account Owner
Initial Submitters	Account Owner
Created By	Pawan Rajak, 9/17/2025, 6:15 AM
Modified By	Pawan Rajak, 9/17/2025, 6:15 AM

4. The approval request is automatically assigned to the submitter's manager via the Role Hierarchy.
5. The Email Alert was added to the "Initial Submission Actions," ensuring the approver is notified instantly.

The screenshot shows the Salesforce Setup interface, specifically the 'Initial Submission Actions' section for the 'Job Posting: Job Posting Approval' process. The 'Initial Submission Actions' table lists two actions: 'Record Lock' and 'Email Alert'. The 'Email Alert' action is highlighted with a red box. Below the table, the 'Approval Steps' section shows a single step named 'Step 1' with a description 'Step 1' and an assigned approver 'User:TPO officer'.

Action	Type	Description
Record Lock	Record Lock	Lock the record from being edited
Email Alert	Email Alert	Notifies Approver of new Job Posting

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Step 1	1	Step 1			User:TPO officer	Final Rejection

6. Field Updates were configured to automatically change the Status to "Open" upon approval or "Needs Rework" upon rejection.


Approval Processes

Action	Type	Description
	Record Lock	Lock the record from being edited

Approval Steps

Action	Step Number	Name	Description	Criteria	Assigned Approver	Reject Behavior
Show Actions Edit	1	Step 1			User:TPO officer	Final Rejection

Final Approval Actions

[Add Existing](#)
[Add New](#)

Action	Type	Description
Edit	Record Lock	Lock the record from being edited
Edit Remove	Field Update	Update Status to Open

Final Rejection Actions

[Add Existing](#)
[Add New](#)

Action	Type	Description
Edit	Record Lock	Unlock the record for editing
Edit Remove	Field Update	Update Status to Needs Rework

Recall Actions

[Add Existing](#)
[Add New](#)

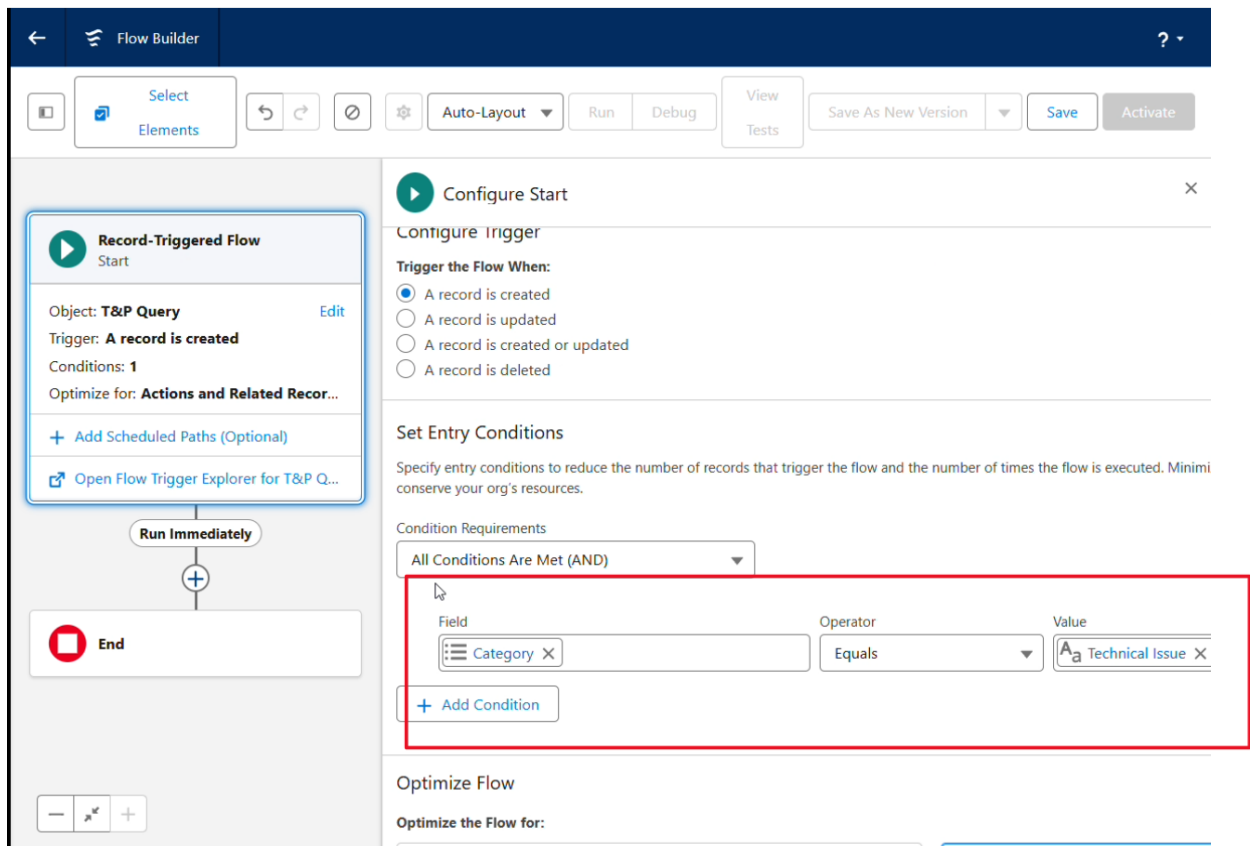
Action	Type	Description
	Record Lock	Unlock the record for editing

[Back To Top](#)
Always show me [more](#) records per related list

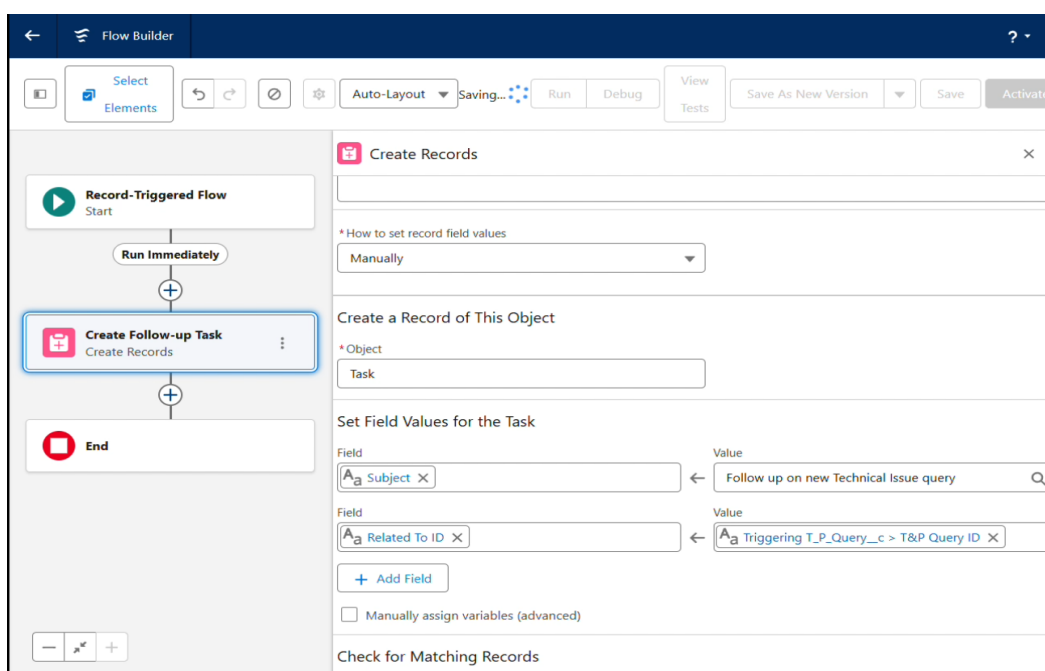
D. Record-Triggered Flow: Automated Task Creation

A background automation was created to ensure that critical student queries are assigned and tracked efficiently.

- **Business Need:** Queries related to technical issues need to be flagged and assigned for follow-up so they are not lost in the general queue.
- **Implementation Steps (Simplified):**
 1. A **Record-Triggered Flow** was built on the T&P Query object that runs only when a new record is created with a Category of "Technical Issue".



2. A Create Records element generates a new Task record.
3. To ensure clarity and trackability, the Task's Subject is set to a static text ("Follow up on new Technical Issue query"), and its Related To ID is dynamically set to the ID of the T&P Query record that triggered the flow ({!\$Record.Id}). This creates a direct link between the query and the follow-up task.



3. Strategically Excluded Concepts & Justification

- **Legacy Tools (Workflow Rules & Process Builder):** These older automation tools were intentionally not used.
 - **Justification:** Salesforce is actively retiring these tools in favor of Flow Builder. To ensure the application is modern, efficient, and future-proof, all automation was built in Flow. This aligns with industry best practices and simplifies future maintenance.
- **Custom Notifications (In-App Alerts):** The in-app notification feature was deferred for this phase.
 - **Justification:** After analyzing the primary user workflows, it was determined that email is the most critical and reliable communication channel for T&P Officers, who may not be logged into Salesforce at all times. To avoid "notification fatigue" and ensure that high-priority alerts (like an approval request) are never missed, the decision was made to centralize these actionable notifications in the email channel for this initial build. In-app notifications are a candidate for a future enhancement phase, pending user feedback on communication preferences.

Phase 5: Apex Programming Implementation Report

Date: September 19, 2025

1. Overview

This phase involved extending the application's capabilities beyond declarative automation by implementing custom server-side logic with Apex. The primary goal was to enforce a complex business rule that required cross-object data validation, a scenario where an Apex Trigger is the most suitable and efficient tool. The implementation followed Salesforce best practices, including bulkification and the mandatory creation of a comprehensive test class.

2. Business Requirement & Technical Solution

- **Business Need:** To maintain a high quality of student applications and manage company expectations, the T&P Cell must ensure that students only apply for jobs where they meet the minimum academic criteria. Specifically, a student must be prevented from applying for a Job Posting if their GPA is lower than the Minimum GPA required for that role.
- **Chosen Solution:** An **Apex Trigger** on the Application__c object. A standard Validation Rule is not ideal for this scenario as it would require complex cross-object formulas. An Apex Trigger provides the necessary power and flexibility to query related records and enforce the logic cleanly before the application is saved.

3. Implementation Details

A. Prerequisite: GPA Field

To enable the validation, a new custom field was added:

- **Object:** Contact
- **Field Label:** GPA
- **Data Type:** Number(2, 2)
- **Purpose:** To store the official GPA for each student record.

B. Apex Trigger: ApplicationTrigger.trigger

A before insert trigger was developed to run before any new Application__c record is committed to the database.

- **Logic and Best Practices Followed:**
 1. **Bulkification:** The trigger first collects all Job_Posting__c and Student__c IDs from the incoming applications (Trigger.new) into Sets. This best practice

ensures the trigger can process a single record or hundreds of records in one transaction without hitting governor limits.

2. **No SOQL in Loops:** Two efficient SOQL queries are performed *outside* of any loops to retrieve all necessary Job Posting and Contact data at once. The results are stored in Maps for quick and easy data retrieval.
3. **Validation Logic:** The trigger then iterates through the new applications. For each application, it retrieves the corresponding Job Posting and Student records from the Maps. An if statement (Control Statement) then compares the student's GPA__c to the job's Minimum_GPA__c.
4. **Error Handling:** If a student's GPA is too low, the addError() method is called on that specific application record. This action stops the save process for that record and displays a clear, user-friendly error message on the screen.

Full Trigger Code:

```
1  /**
2   * @description Trigger on the Application object to handle custom validation.
3   */
4  trigger ApplicationTrigger on Application__c (before insert) {
5
6      // Best Practice: Create a set to hold the IDs of all Job Postings and Contacts from the new applications.
7      // A Set is a collection that does not allow duplicate values, which is efficient for queries.
8      Set<Id> jobPostingIds = new Set<Id>();
9      Set<Id> studentIds = new Set<Id>();
10
11     // 1. COLLECT IDs: Loop through all the new Application records that are trying to be created.
12     // Trigger.new is a special context variable that holds all the new records in the transaction.
13     for (Application__c app : Trigger.new) {
14         jobPostingIds.add(app.Job_Posting__c);
15         studentIds.add(app.Student__c);
16     }
17
18     // 2. QUERY DATA: Use SOQL to get all the required data in one efficient query for Job Postings
19     // and one for Contacts. This avoids running queries inside a loop, which is a major performance issue.
20     Map<Id, Job_Posting__c> jobPostingsMap = new Map<Id, Job_Posting__c>([
21         SELECT Id, Minimum_GPA__c FROM Job_Posting__c WHERE Id IN :jobPostingIds
22     ]);
23
24     Map<Id, Contact> studentsMap = new Map<Id, Contact>([
25         SELECT Id, GPA__c FROM Contact WHERE Id IN :studentIds
26     ]);
27
28     // 3. VALIDATE LOGIC: Loop through the new applications again. This time, we have all the data we need.
29     for (Application__c app : Trigger.new) {
30         // Retrieve the specific Job Posting and Student for the current application from our Maps.
31         Job_Posting__c relatedJob = jobPostingsMap.get(app.Job_Posting__c);
32         Contact relatedStudent = studentsMap.get(app.Student__c);
33
34         // Control Statement: Check if the student's GPA is less than the job's minimum requirement.
35         // Also check that both values are not null to prevent errors.
36         if (relatedStudent.GPA__c != null &&
37             relatedJob.Minimum_GPA__c != null &&
38             relatedStudent.GPA__c < relatedJob.Minimum_GPA__c) {
39
40             // If the GPA is too low, add a custom error message to the record.
41             // This prevents the record from being saved and displays the error to the user.
42             app.addError('You cannot apply for this job. Your GPA (' + relatedStudent.GPA__c + ') is below the minimum requirement of ' + relatedJob.Minimum_GPA__c + '.');
43         }
44     }
45 }
```

C. Apex Test Class: ApplicationTriggerTest.cls

A dedicated test class was written to validate the trigger's functionality and ensure it is ready for deployment. The class achieved 100% code coverage.

- **Test Setup (@testSetup):** A setup method was used to create all necessary test data in a single transaction. This included a test Company, a Job Posting with a high GPA requirement (9.0), a Student with a qualifying GPA (9.5), and a Student with a non-qualifying GPA (8.0).
- **Positive Test Scenario:** A test method verifies that the studentWithHighGPA can successfully create an Application record. The test passes by using `System.assertEquals()` to confirm that one application record was created.
- **Negative Test Scenario:** A test method verifies that the studentWithLowGPA is blocked from creating an Application. This is tested by wrapping the insert statement in a try-catch block, asserting that the system throws a `DmlException` and that the error message contains the expected text from the trigger.
- Full Test Code:

```

1  /**
2   * @description Test class for the ApplicationTrigger.
3   */
4  @isTest
5  private class ApplicationTriggerTest {
6
7      // @testSetup is a special method that runs once to create all the test data needed by all test methods.
8      @testSetup
9      static void setup() {
10         // Create a test Company
11         Account testCompany = new Account(Name = 'Test Corp');
12         insert testCompany;
13
14         // Create a test Job Posting that requires a high GPA
15         Job_Posting__c testJob = new Job_Posting__c(
16             Name = 'Senior Developer',
17             Company__c = testCompany.Id,
18             Minimum_GPA__c = 9.0
19         );
20         insert testJob;
21
22         // Create two test Students (Contacts)
23         Contact studentWithHighGPA = new Contact(
24             LastName = 'Green',
25             RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfoByName().get('Student').getRecordTypeId(),
26             GPA__c = 9.5
27         );
28         Contact studentWithLowGPA = new Contact(
29             LastName = 'Red',
30             RecordTypeId = Schema.SObjectType.Contact.getRecordTypeInfoByName().get('Student').getRecordTypeId(),
31             GPA__c = 8.0
32         );
33         insert new List<Contact>{ studentWithHighGPA, studentWithLowGPA };
34     }
35
36     // A positive test method to verify that a student with a high GPA CAN apply.
37     @isTest
38     static void testApplication_PositiveScenario() {
39         // Get the test data we created in the setup method.
40         Job_Posting__c job = [SELECT Id FROM Job_Posting__c LIMIT 1];
41         Contact student = [SELECT Id FROM Contact WHERE LastName = 'Green' LIMIT 1];
42
43         // Start the test. This creates a fresh set of governor limits.
44         Test.startTest();
45
46         // Perform the action: Create an Application record.
47         Application__c app = new Application__c(
48             Job_Posting__c = job.Id,
49             Student__c = student.Id
50         );
51         insert app;
52
53         // Stop the test.
54         Test.stopTest();
55
56         // Assertion: Verify that the application was successfully created by querying it.
57         // If the record exists, the test passes.
58         List<Application__c> createdApps = [SELECT Id FROM Application__c WHERE Id = :app.Id];
59         System.assertEquals(1, createdApps.size(), 'Application should have been created successfully.');
```

4. Strategically Excluded Concepts & Justification

- **Asynchronous Apex (Batch, Queueable, Future, Scheduled):** The business requirement is for immediate, real-time validation at the moment a user tries to save a record. Asynchronous processing, which runs in the background, is not suitable for this use case.
- **Trigger Design Patterns (Handler Classes):** While using a separate handler class is a best practice for complex orgs with multiple triggers on one object, it constitutes over-engineering for this project's scope. The implemented trigger is self-contained and already follows the most critical design principles: bulkification and no DML/SOQL in loops.
- **SOSL (Salesforce Object Search Language):** SOSL is a tool for text-based searches across multiple objects (like a search engine). The trigger's requirement was to fetch specific records based on known IDs, for which SOQL is the correct and more performant tool.